

CS3014 - Estructuras de Datos Avanzadas

Fractional Cascading II

Semana 5

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

9 de septiembre de 2026

Objetivos de la sesión



Una sola pregunta, toda la clase

¿Cómo consultar una caja $[x_1, x_2] \times [y_1, y_2] \times [z_1, z_2]$ entre n puntos en $O(\lg n + k)$ — la **misma** complejidad que en 2D?

Objetivos de la sesión



Una sola pregunta, toda la clase

¿Cómo consultar una caja $[x_1, x_2] \times [y_1, y_2] \times [z_1, z_2]$ entre n puntos en $O(\lg n + k)$ — la **misma** complejidad que en 2D?

- Por qué la idea “obvia” de cascadear dos veces no alcanza

Objetivos de la sesión



Una sola pregunta, toda la clase

¿Cómo consultar una caja $[x_1, x_2] \times [y_1, y_2] \times [z_1, z_2]$ entre n puntos en $O(\lg n + k)$ — la **misma** complejidad que en 2D?

- ▶ Por qué la idea “obvia” de cascadear dos veces no alcanza
- ▶ El caso base escondido: una consulta de **dominancia** en 2D, resuelta con geometría (rayos y una subdivisión del plano)

Objetivos de la sesión



Una sola pregunta, toda la clase

¿Cómo consultar una caja $[x_1, x_2] \times [y_1, y_2] \times [z_1, z_2]$ entre n puntos en $O(\lg n + k)$ — la **misma** complejidad que en 2D?

- ▶ Por qué la idea “obvia” de cascadear dos veces no alcanza
- ▶ El caso base escondido: una consulta de **dominancia** en 2D, resuelta con geometría (rayos y una subdivisión del plano)
- ▶ El truco para no pagar $O(\lg n)$ extra por cada dimensión nueva: caminar a un único **nodo de separación**

Objetivos de la sesión



Una sola pregunta, toda la clase

¿Cómo consultar una caja $[x_1, x_2] \times [y_1, y_2] \times [z_1, z_2]$ entre n puntos en $O(\lg n + k)$ — la **misma** complejidad que en 2D?

- ▶ Por qué la idea “obvia” de cascadear dos veces no alcanza
- ▶ El caso base escondido: una consulta de **dominancia** en 2D, resuelta con geometría (rayos y una subdivisión del plano)
- ▶ El truco para no pagar $O(\lg n)$ extra por cada dimensión nueva: caminar a un único **nodo de separación**
- ▶ *Fractional cascading* generalizado, uniendo todo

Motivación

Repaso relámpago: 2D



Lo que ya sabemos

Un **Layered Range Tree** en 2D responde consultas $[x_1, x_2] \times [y_1, y_2]$ en $O(\lg n + k)$: una búsqueda binaria real en la raíz, y luego $O(1)$ por nivel gracias a los puentes del *fractional cascading*.

Repaso relámpago: 2D



Lo que ya sabemos

Un **Layered Range Tree** en 2D responde consultas $[x_1, x_2] \times [y_1, y_2]$ en $O(\lg n + k)$: una búsqueda binaria real en la raíz, y luego $O(1)$ por nivel gracias a los puentes del *fractional cascading*.

Hoy: ¿qué pasa si agregamos una tercera coordenada z ?

¡Piénsalo!



Antes de continuar

Con árboles anidados **sin** ningún *cascading*, ¿qué complejidad esperarías para una caja 3D? ¿Y si aplicamos *layered range tree* **solo** en el nivel más interno (2D), y arriba usamos árboles normales?

¡Piénsalo!



Antes de continuar

Con árboles anidados **sin** ningún *cascading*, ¿qué complejidad esperarías para una caja 3D? ¿Y si aplicamos *layered range tree* **solo** en el nivel más interno (2D), y arriba usamos árboles normales?

Ingenuo (3 BSTs anidados): $O(\lg^3 n + k)$. *Cascading* solo en la base: $O(\lg^2 n + k)$ (un factor $\lg n$ por cada nivel que **no** está cascadeado). ¿Se puede llegar a $O(\lg n + k)$?

La trampa: “cascadear dos veces” no alcanza



Una idea tentadora — pero incompleta

Podrías pensar: cuelgo, de cada nodo del árbol en x , un arreglo- y cascadeado **y** un arreglo- z cascadeado, **independientes**. Cascadear te da la posición de y_1 y la de z_1 por separado en $O(1)$ al bajar — pero **no** te dice cuántos puntos cumplen **ambas** condiciones **a la vez**.

La trampa: “cascadear dos veces” no alcanza



Una idea tentadora — pero incompleta

Podrías pensar: cuelgo, de cada nodo del árbol en x , un arreglo- y cascadeado **y** un arreglo- z cascadeado, **independientes**. Cascadear te da la posición de y_1 y la de z_1 por separado en $O(1)$ al bajar — pero **no** te dice cuántos puntos cumplen **ambas** condiciones **a la vez**.

Contar la intersección de dos condiciones independientes no es gratis: perderías la relación entre “quién cumple y ” y “quién cumple z ”.

Reformulando el problema



La observación clave

Dentro de cada nodo del árbol en x , el problema que de verdad hay que resolver es: dado un conjunto de puntos (y, z) , reportar los que cumplen $y \leq b_2$ y $z \leq b_3$ **simultáneamente**. Esto es una consulta de **dominancia** en 2D — un problema geométrico con nombre propio, y una solución propia.

Reformulando el problema



La observación clave

Dentro de cada nodo del árbol en x , el problema que de verdad hay que resolver es: dado un conjunto de puntos (y, z) , reportar los que cumplen $y \leq b_2$ y $z \leq b_3$ **simultáneamente**. Esto es una consulta de **dominancia** en 2D — un problema geométrico con nombre propio, y una solución propia.

Si resolvemos dominancia en $O(\lg n + k)$, el resto es “solo” encajarla dentro de árboles en x , y y z sin perder esa complejidad.

El caso base: dominancia en 2D

El problema de dominancia



Definición

Dados n puntos (y_i, z_i) y una consulta (b_2, b_3) , reportar todos los puntos con $y_i \leq b_2$ y $z_i \leq b_3$. Queremos $O(\lg n + k)$ tiempo, $O(n)$ espacio.

El problema de dominancia



Definición

Dados n puntos (y_i, z_i) y una consulta (b_2, b_3) , reportar todos los puntos con $y_i \leq b_2$ y $z_i \leq b_3$. Queremos $O(\lg n + k)$ tiempo, $O(n)$ espacio.

¡Piénsalo!

Un Layered Range Tree normal ya resuelve esto en $O(\lg n + k)$. ¿Por qué molestarnos con una estructura distinta?

El problema de dominancia



Definición

Dados n puntos (y_i, z_i) y una consulta (b_2, b_3) , reportar todos los puntos con $y_i \leq b_2$ y $z_i \leq b_3$. Queremos $O(\lg n + k)$ tiempo, $O(n)$ espacio.

¡Piénsalo!

Un Layered Range Tree normal ya resuelve esto en $O(\lg n + k)$. ¿Por qué molestarnos con una estructura distinta?

Porque en un momento vamos a necesitar **muchas** copias de esta estructura, una por nodo de otro árbol — y necesitamos que el espacio total siga siendo $O(n)$ **por copia**, con una forma de **combinarlas** más adelante. Una estructura puramente geométrica (sin árbol anidado) es más fácil de acoplar después.

Reformulando como rayos



El truco geométrico

Cada punto (y_i, z_i) se convierte en un **rayo vertical** que sale de (y_i, z_i) hacia arriba (hacia $z = +\infty$). La consulta (b_2, b_3) se convierte en un **rayo horizontal** que sale de (b_2, b_3) hacia la izquierda.

Reformulando como rayos

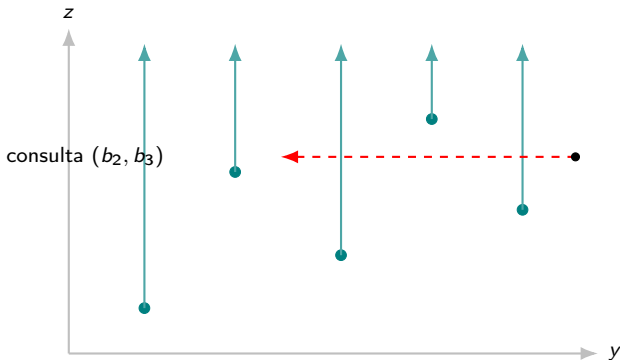


El truco geométrico

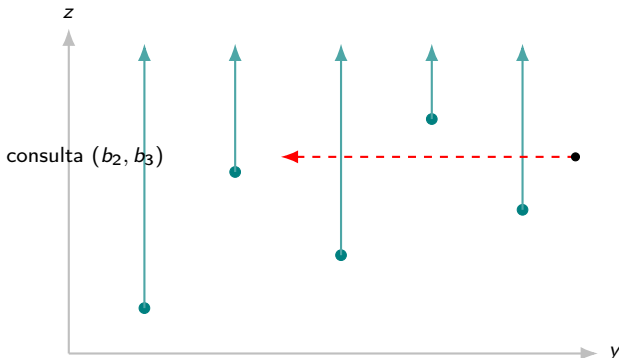
Cada punto (y_i, z_i) se convierte en un **rayo vertical** que sale de (y_i, z_i) hacia arriba (hacia $z = +\infty$). La consulta (b_2, b_3) se convierte en un **rayo horizontal** que sale de (b_2, b_3) hacia la izquierda.

Un punto cumple $y_i \leq b_2 \wedge z_i \leq b_3$ **si y solo si** su rayo vertical es **atravesado** por el rayo horizontal de la consulta.

Visualizando los rayos



Visualizando los rayos



El rayo de la consulta atraviesa los rayos de los puntos con $y \leq b_2$ (a su izquierda) y $z \leq b_3$ (por debajo de su altura): exactamente los puntos dominados.

Construcción: una subdivisión del plano



La idea

Se traza un segmento horizontal a partir de cada punto. Para mantener el espacio en $O(n)$, **no** todos los segmentos llegan hasta el final: algunos se **ex-tienden** y otros se **cortan**, usando la misma idea de “promover la mitad” que ya vimos en *fractional cascading*, hasta lograr que cada cara de la subdivisión resultante tenga **grado acotado** (un número constante de vecinos).

Construcción: una subdivisión del plano



La idea

Se traza un segmento horizontal a partir de cada punto. Para mantener el espacio en $O(n)$, **no** todos los segmentos llegan hasta el final: algunos se **ex-tienden** y otros se **cortan**, usando la misma idea de “promover la mitad” que ya vimos en *fractional cascading*, hasta lograr que cada cara de la subdivisión resultante tenga **grado acotado** (un número constante de vecinos).

Este es el resultado geométrico no trivial de Chazelle (1986): una subdivisión de grado acotado con $O(n)$ segmentos en total. No lo derivaremos en detalle aquí — lo que sí necesitamos entender bien es **cómo se consulta**.

La consulta: buscar y luego caminar



Dos pasos

La consulta: buscar y luego caminar



Dos pasos

1. Una **única** búsqueda binaria real, para ubicar b_3 entre los segmentos de la columna más a la izquierda: $O(\lg n)$.

La consulta: buscar y luego caminar



Dos pasos

1. Una **única** búsqueda binaria real, para ubicar b_3 entre los segmentos de la columna más a la izquierda: $O(\lg n)$.
2. Desde ahí, **caminar hacia la derecha** por las caras de la subdivisión, en $O(1)$ por cara, hasta llegar a la posición horizontal b_2 .

La consulta: buscar y luego caminar



Dos pasos

1. Una **única** búsqueda binaria real, para ubicar b_3 entre los segmentos de la columna más a la izquierda: $O(\lg n)$.
2. Desde ahí, **caminar hacia la derecha** por las caras de la subdivisión, en $O(1)$ por cara, hasta llegar a la posición horizontal b_2 .

El reporte es gratis

Cada rayo vertical que el camino **cruza** en el paso 2 es exactamente uno de los k puntos dominados — el recorrido **es** el mecanismo de reporte, no un paso aparte.

Pseudocódigo de la consulta



Algoritmo 1: CONSULTADOMINANCIA(S, b_2, b_3)

cara $\leftarrow$ búsqueda binaria de b_3 en la columna izquierda de S // $O(\lg n)$

resultado $\leftarrow$ lista vacía

mientras *la posición horizontal de cara es* $< b_2$ **hacer**

si *el borde de cara es un rayo de un punto* **entonces**

 agregar ese punto a *resultado*

cara $\leftarrow$ cara vecina hacia la derecha // $O(1)$

devolver *resultado*

Pseudocódigo de la consulta



Algoritmo 2: CONSULTADOMINANCIA(S, b_2, b_3)

cara $\leftarrow$ búsqueda binaria de b_3 en la columna izquierda de S // $O(\lg n)$

resultado $\leftarrow$ lista vacía

mientras *la posición horizontal de cara es* $< b_2$ **hacer**

si *el borde de cara es un rayo de un punto entonces*

 agregar ese punto a *resultado*

cara $\leftarrow$ cara vecina hacia la derecha // $O(1)$

devolver *resultado*

Costo total: $O(\lg n)$ de la búsqueda inicial, más $O(1)$ por cada cara visitada. Como se visita exactamente una cara por punto reportado (más un puñado de caras “vacías”), el total es $O(\lg n + k)$.

Agregando dimensiones acotadas

Paso 1: acotar x



Estructura $D_2: [x_1, x_2] \times (-\infty, b_2] \times (-\infty, b_3)$

Un BST estándar sobre x . Cada nodo guarda, para los puntos de su subárbol, una copia de la estructura de dominancia (D_1) que acabamos de construir.

Paso 1: acotar x



Estructura $D_2: [x_1, x_2] \times (-\infty, b_2] \times (-\infty, b_3)$

Un BST estándar sobre x . Cada nodo guarda, para los puntos de su subárbol, una copia de la estructura de dominancia (D_1) que acabamos de construir.

La consulta usa la descomposición canónica de siempre: los $O(\lg n)$ nodos canónicos de $[x_1, x_2]$, disparando una consulta de dominancia en cada uno. Nada nuevo todavía — lo interesante empieza al agregar la **segunda** dimensión acotada.

¡Piénsalo!



El peligro de agregar y acotado

Si ahora acotamos también $y \in [a_2, b_2]$ de la forma obvia (otra descomposición canónica, esta vez sobre y), ¿cuántas consultas de dominancia terminarías disparando en total?

¡Piénsalo!



El peligro de agregar y acotado

Si ahora acotamos también $y \in [a_2, b_2]$ de la forma obvia (otra descomposición canónica, esta vez sobre y), ¿cuántas consultas de dominancia terminarías disparando en total?

$O(\lg n)$ nodos canónicos en x , cada uno con $O(\lg n)$ nodos canónicos en y :
 $O(\lg^2 n)$ consultas. Perdimos el $O(\lg n)$ que queríamos. Necesitamos evitar la descomposición canónica en y por completo.

Paso 2: acotar y — el nodo de separación



Estructura D_3 : $[x_1, x_2] \times [a_2, b_2] \times (-\infty, b_3)$

Un **árbol de rango** sobre y (no sobre x). Cada nodo v de este árbol guarda dos estructuras satélite, sobre los puntos de sus dos hijos:

Paso 2: acotar y — el nodo de separación



Estructura $D_3: [x_1, x_2] \times [a_2, b_2] \times (-\infty, b_3)$

Un **árbol de rango sobre y** (no sobre x). Cada nodo v de este árbol guarda dos estructuras satélite, sobre los puntos de sus dos hijos:

- ▶ en derecha(v): una estructura D_2 normal (acotada por arriba, $y \leq b_2$);

Paso 2: acotar y — el nodo de separación



Estructura $D_3: [x_1, x_2] \times [a_2, b_2] \times (-\infty, b_3)$

Un **árbol de rango sobre y** (no sobre x). Cada nodo v de este árbol guarda dos estructuras satélite, sobre los puntos de sus dos hijos:

- ▶ en derecha(v): una estructura D_2 normal (acotada por arriba, $y \leq b_2$);
- ▶ en izquierda(v): una estructura D_2 **invertida**, D'_2 (acotada por abajo, $y \geq a_2$).

¿Por qué alcanza con un solo nodo?



El nodo de separación (*split node*)

Al buscar a_2 y b_2 en el árbol de y , sus caminos coinciden hasta un nodo v_{split} donde se separan —exactamente como en un árbol de rangos 1D clásico.

¿Por qué alcanza con un solo nodo?



El nodo de separación (*split node*)

Al buscar a_2 y b_2 en el árbol de y , sus caminos coinciden hasta un nodo v_{split} donde se separan —exactamente como en un árbol de rangos 1D clásico.

- ▶ Todo derecha(v_{split}) ya cumple $y \geq a_2$ automáticamente: solo falta pedir $y \leq b_2 \Rightarrow$ una consulta a su D_2 .

¿Por qué alcanza con un solo nodo?



El nodo de separación (*split node*)

Al buscar a_2 y b_2 en el árbol de y , sus caminos coinciden hasta un nodo v_{split} donde se separan —exactamente como en un árbol de rangos 1D clásico.

- ▶ Todo derecha(v_{split}) ya cumple $y \geq a_2$ automáticamente: solo falta pedir $y \leq b_2 \Rightarrow$ una consulta a su D_2 .
- ▶ Todo izquierda(v_{split}) ya cumple $y \leq b_2$ automáticamente: solo falta pedir $y \geq a_2 \Rightarrow$ una consulta a su D'_2 (invertida).

¿Por qué alcanza con un solo nodo?



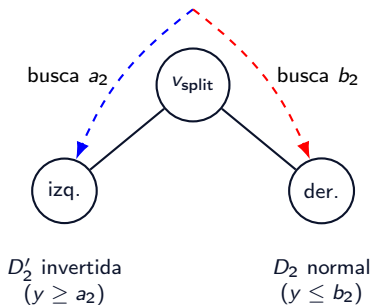
El nodo de separación (*split node*)

Al buscar a_2 y b_2 en el árbol de y , sus caminos coinciden hasta un nodo v_{split} donde se separan —exactamente como en un árbol de rangos 1D clásico.

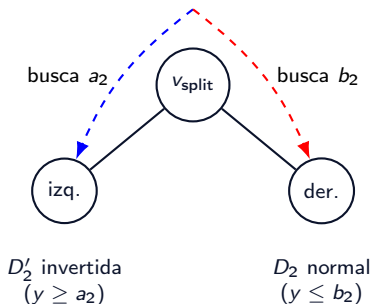
- ▶ Todo derecha(v_{split}) ya cumple $y \geq a_2$ automáticamente: solo falta pedir $y \leq b_2 \Rightarrow$ una consulta a su D_2 .
- ▶ Todo izquierda(v_{split}) ya cumple $y \leq b_2$ automáticamente: solo falta pedir $y \geq a_2 \Rightarrow$ una consulta a su D'_2 (invertida).

Dos consultas, en un único nodo — no $O(\lg n)$ nodos canónicos.

Visualizando el nodo de separación



Visualizando el nodo de separación



Los caminos de a_2 y b_2 se separan en v_{split} ; de ahí en adelante ya no hace falta seguir bajando.

Paso 3: acotar z — la misma idea, un nivel más



Estructura D_4 : $[x_1, x_2] \times [a_2, b_2] \times [a_3, b_3]$

Exactamente la misma construcción que D_3 , pero sobre z en vez de y : un árbol de rango sobre z , cuyo nodo de separación guarda una estructura D_3 normal a un lado y una D_3 invertida al otro.

Paso 3: acotar z — la misma idea, un nivel más



Estructura $D_4: [x_1, x_2] \times [a_2, b_2] \times [a_3, b_3]$

Exactamente la misma construcción que D_3 , pero sobre z en vez de y : un árbol de rango sobre z , cuyo nodo de separación guarda una estructura D_3 normal a un lado y una D_3 invertida al otro.

Sin cascading todavía

Tal como está, cada nivel (D_3 dentro de D_4) todavía hace su propia búsqueda binaria real: $O(\lg n) \cdot O(\lg n) + k = O(\lg^2 n + k)$. Falta la pieza final.

Fractional Cascading generalizado

La pieza final



El problema que queda

Tenemos D_1 , D_2 , D_3 , D_4 correctamente **anidadas** entre sí, pero cada nivel todavía **re-busca desde cero** en el nivel de abajo. Ya vimos cómo evitar exactamente eso con *fractional cascading* — solo que ahora las “listas” no son simples arreglos, sino estructuras enteras con más de un vecino.

La pieza final



El problema que queda

Tenemos D_1, D_2, D_3, D_4 correctamente **anidadas** entre sí, pero cada nivel todavía **re-busca desde cero** en el nivel de abajo. Ya vimos cómo evitar exactamente eso con *fractional cascading* — solo que ahora las “listas” no son simples arreglos, sino estructuras enteras con más de un vecino.

Necesitamos generalizar *fractional cascading* de “una cadena de listas” a “un grafo de estructuras”.

El grafo de *cascading* generalizado



Idea general (recordando la de listas)

Fractional cascading funciona en cualquier **grafo** donde cada vértice guarda un conjunto de elementos, las aristas están etiquetadas con boletos hacia elementos vecinos, y **cada vértice tiene grado acotado** (un número constante de vecinos). Bajo esas condiciones, la navegación completa cuesta $O(k + \lg n)$: una búsqueda real al entrar, y $O(1)$ por cada paso.

El grafo de *cascading* generalizado



Idea general (recordando la de listas)

Fractional cascading funciona en cualquier **grafo** donde cada vértice guarda un conjunto de elementos, las aristas están etiquetadas con boletos hacia elementos vecinos, y **cada vértice tiene grado acotado** (un número constante de vecinos). Bajo esas condiciones, la navegación completa cuesta $O(k + \lg n)$: una búsqueda real al entrar, y $O(1)$ por cada paso.

En nuestro caso, cada nodo del árbol en D_4/D_3 tiene grado **acotado por 5**: su padre, sus dos hijos, y dos enlaces auxiliares hacia la estructura invertida gemela del mismo nodo. Eso basta para aplicar la misma idea.

Componiendo todo



El resultado

Con el grafo de *cascading* generalizado conectando D_4 sobre D_3 sobre D_2 sobre D_1 , **una sola** búsqueda binaria real al entrar por x **alcanza** para todos los niveles: cada paso hacia abajo (incluyendo los saltos entre estructura normal e invertida) es $O(1)$.

Componiendo todo



El resultado

Con el grafo de *cascading* generalizado conectando D_4 sobre D_3 sobre D_2 sobre D_1 , **una sola** búsqueda binaria real al entrar por x **alcanza** para todos los niveles: cada paso hacia abajo (incluyendo los saltos entre estructura normal e invertida) es $O(1)$.

$$\underbrace{O(\lg n)}_{\text{una búsqueda real}} + \underbrace{O(1) \cdot O(\lg n)}_{\text{pasos de descenso}} + \underbrace{O(k)}_{\text{reporte}} = O(\lg n + k)$$

Componiendo todo



El resultado

Con el grafo de *cascading* generalizado conectando D_4 sobre D_3 sobre D_2 sobre D_1 , **una sola** búsqueda binaria real al entrar por x **alcanza** para todos los niveles: cada paso hacia abajo (incluyendo los saltos entre estructura normal e invertida) es $O(1)$.

$$\underbrace{O(\lg n)}_{\text{una búsqueda real}} + \underbrace{O(1) \cdot O(\lg n)}_{\text{pasos de descenso}} + \underbrace{O(k)}_{\text{reporte}} = O(\lg n + k)$$

La misma complejidad que 2D — ganamos una dimensión prácticamente gratis, a cambio de una construcción mucho más cuidadosa.

Resultado y generalización

¿Cuánto cuesta en espacio?



El precio de tanta estructura anidada

Cada punto aparece, en promedio, $O(\lg^3 n)$ veces entre las distintas copias de D_1 , D_2 y D_3 anidadas dentro de D_4 . El espacio total es $O(n \lg^3 n)$ — más que los $O(n)$ de 2D, pero polinomial en $\lg n$, no en n .

¿Cuánto cuesta en espacio?



El precio de tanta estructura anidada

Cada punto aparece, en promedio, $O(\lg^3 n)$ veces entre las distintas copias de D_1 , D_2 y D_3 anidadas dentro de D_4 . El espacio total es $O(n \lg^3 n)$ — más que los $O(n)$ de 2D, pero polinomial en $\lg n$, no en n .

Trade-off típico de estructuras geométricas avanzadas: se paga espacio extra (logarítmico) por tiempo de consulta óptimo.

Comparando las tres versiones



Técnica	Consulta	Espacio
----------------	-----------------	----------------

Comparando las tres versiones



Técnica	Consulta	Espacio
BSTs anidados (sin <i>cascading</i>)	$O(\lg^3 n + k)$	$O(n)$

Comparando las tres versiones



Técnica	Consulta	Espacio
BSTs anidados (sin <i>cascading</i>)	$O(\lg^3 n + k)$	$O(n)$
<i>Cascading</i> solo en la base 2D	$O(\lg^2 n + k)$	$O(n \lg n)$

Comparando las tres versiones



Técnica	Consulta	Espacio
BSTs anidados (sin <i>cascading</i>)	$O(\lg^3 n + k)$	$O(n)$
<i>Cascading</i> solo en la base 2D	$O(\lg^2 n + k)$	$O(n \lg n)$
<i>Cascading</i> completo (D_1 - D_4)	$O(\lg n + k)$	$O(n \lg^3 n)$

Comparando las tres versiones



Técnica	Consulta	Espacio
BSTs anidados (sin <i>cascading</i>)	$O(\lg^3 n + k)$	$O(n)$
<i>Cascading</i> solo en la base 2D	$O(\lg^2 n + k)$	$O(n \lg n)$
<i>Cascading</i> completo (D_1 - D_4)	$O(\lg n + k)$	$O(n \lg^3 n)$

Cada nivel adicional de *cascading* bien aplicado quita un factor de $\lg n$ del tiempo de consulta, al costo de un factor de $\lg n$ extra en espacio.

El patrón general en d dimensiones



Generalizando (Chazelle & Guibas, 1986)

Para $d \geq 3$ dimensiones, con *fractional cascading* completo (no solo en la base): consulta $O(\lg^{d-2} n + k)$, espacio $O(n \lg^{d-1} n)$.

El patrón general en d dimensiones



Generalizando (Chazelle & Guibas, 1986)

Para $d \geq 3$ dimensiones, con *fractional cascading* completo (no solo en la base): consulta $O(\lg^{d-2} n + k)$, espacio $O(n \lg^{d-1} n)$.

Sin cascading completo

Layered range tree aplicando *cascading* solo en el nivel más interno (Willard 1985; Gabow, Bentley & Tarjan 1984): consulta $O(\lg^{d-1} n + k)$, espacio $O(n \lg^{d-1} n)$.

El patrón general en d dimensiones



Generalizando (Chazelle & Guibas, 1986)

Para $d \geq 3$ dimensiones, con *fractional cascading* completo (no solo en la base): consulta $O(\lg^{d-2} n + k)$, espacio $O(n \lg^{d-1} n)$.

Sin cascading completo

Layered range tree aplicando *cascading* solo en el nivel más interno (Willard 1985; Gabow, Bentley & Tarjan 1984): consulta $O(\lg^{d-1} n + k)$, espacio $O(n \lg^{d-1} n)$.

Para $d = 3$: $\lg^{d-2} n = \lg n$ (lo que acabamos de construir) contra $\lg^{d-1} n = \lg^2 n$ (la versión más simple).

Cierre de la sesión



Cierre de la sesión



- ▶ El error a evitar: cascadear y y z **por separado** no basta, porque no captura la condición **conjunta** $y \leq b_2 \wedge z \leq b_3$.
- ▶ El problema real, escondido en cada nodo del árbol en x , es una consulta de **dominancia** 2D — resuelta reformulando puntos como rayos y consultas como rayos, con una subdivisión del plano de grado acotado.
- ▶ Para acotar y (y luego z) sin pagar $O(\lg n)$ -vías extra, se usa el **nodo de separación** junto con una estructura “gemela invertida” a cada lado.
- ▶ *Fractional cascading generalizado* (grafo de grado acotado, no solo una cadena de listas) une todos los niveles en una sola búsqueda real: $O(\lg n + k)$ consulta, $O(n \lg^3 n)$ espacio.
- ▶ El patrón se extiende a d dimensiones: $O(\lg^{d-2} n + k)$ con *cascading* completo, contra $O(\lg^{d-1} n + k)$ aplicándolo solo en la base.

Logros de la sesión



Logros de la sesión



- ▶ Sabemos explicar por qué cascadear y y z por separado no captura la condición conjunta que 3D necesita.

Logros de la sesión



- ▶ Sabemos explicar por qué cascadear y y z por separado no captura la condición conjunta que 3D necesita.
- ▶ Sabemos resolver una consulta de dominancia 2D con rayos y una subdivisión del plano de grado acotado.

Logros de la sesión



- ▶ Sabemos explicar por qué cascadear y y z por separado no captura la condición conjunta que 3D necesita.
- ▶ Sabemos resolver una consulta de dominancia 2D con rayos y una subdivisión del plano de grado acotado.
- ▶ Sabemos usar el nodo de separación junto con una estructura gemela invertida para acotar y y z sin descomposición canónica.

Logros de la sesión



- ▶ Sabemos explicar por qué cascadear y y z por separado no captura la condición conjunta que 3D necesita.
- ▶ Sabemos resolver una consulta de dominancia 2D con rayos y una subdivisión del plano de grado acotado.
- ▶ Sabemos usar el nodo de separación junto con una estructura gemela invertida para acotar y y z sin descomposición canónica.
- ▶ Sabemos componer todo con *fractional cascading* generalizado para lograr $O(\lg n + k)$ en 3D.

Gracias

Referencias



Bibliografía citada

- ▶ Chazelle, B., & Guibas, L. J. (1986). Fractional cascading: I. A data structuring technique. *Algorithmica*, 1(1-4).
- ▶ Chazelle, B. (1986). Filtering search: A new approach to query-answering. *SIAM Journal on Computing*, 15(3).
- ▶ Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. <https://courses.csail.mit.edu/6.851/spring21/>.
- ▶ de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications*. 3rd ed. Springer.

Agradecimientos



Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.

Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.